NumPy Vector & Matrix Operations

딥러닝 학습을 위한 NumPy 빠른 참조표

Table 1. Array and Shape Operations

연산	NumPy	예시	결과 / 의미
배열 생성	np.array()	x = np.array([1, 2, 3])	1 - D array, shape = (3,)
0 / 1 생성	np.zeros() / np.ones()	np.zeros((2, 3))	지정한 shape의 배열 생성
범위 생성	np.arange()	np.arange(0, 6)	[0, 1, 2, 3, 4, 5]
Shape / 차원	x.shape / x.ndim	W.shape	각 axis의 크기 / axis 개수
Reshape	x.reshape()	x.reshape(- 1, 1)	(n,) - > (n, 1) column vector
Flatten	x.ravel()	W.ravel()	다차원 배열 - > 1 - D array
Transpose	W.T	W.shape = (m, n)	W.T.shape = (n, m)

Table 2. Vector and Matrix Operations

연산	수학식	NumPy	Shape	딥러닝 의미
Element - wise product	a ⊙ b	a * b	(n,) * (n,) - > (n,)	원소별 곱; activation derivative와 gradient 결합
Dot product	w^T x	w @ x	(n,) @ (n,) - > scalar	single neuron의 weighted sum
Matrix - vector product	W x	W @ x	(m,n) @ (n,) - > (m,)	여러 neuron의 선형 계산
Matrix - matrix product	A B	A @ B	(m,n) @ (n,p) - > (m,p)	여러 vector의 선형 변환
Outer product	u v^T	np.outer(u, v)	(m,) outer (n,) - > (m,n)	weight gradient 행렬 구성
L2 norm	||x||_2	np.linalg.norm(x)	(n,) - > scalar	vector 또는 gradient 크기

@ 연산자: vector @ vector = dot product; matrix @ vector = matrix - vector product; matrix @ matrix = matrix product. *는 element - wise multiplication이다.

Table 3. Statistics and Diagnostic Operations

연산	NumPy	딥러닝 진단 활용
Sum / Mean	np.sum(x), np.mean(x)	loss, activation, gradient의 집계와 평균
Standard deviation	np.std(x)	activation / gradient 분포의 퍼짐 정도
Minimum / Maximum	np.min(x), np.max(x)	극단값과 saturation 관찰
Absolute value	np.abs(x)	gradient absolute magnitude
Zero ratio	np.mean(x == 0)	zero activation 또는 zero gradient 비율

Deep Learning Formula Reference

각 수식을 수학식 -> Shape -> NumPy -> 딥러닝 의미 순서로 연결

Table 4. Forward and Backpropagation Formula Map

개념	수학식	Shape	NumPy	딥러닝에서의 의미
Single neuron	z = w^T x + b	w: (n,1), x: (n,1) w^T: (1,n) z: (1,1)	z = w.T @ x + b 1-D: z = w @ x + b	입력별 weight를 적용한 weighted sum에 bias를 더해 선형 입력 z 생성
Activation	a = f(z)	z: (m,) -> a: (m,)	a = f(z)	선형 입력에 비선형 activation function 적용
Layer forward	z = W x + b	W: (m,n), x: (n,1) z: (m,1)	z = W @ x + b	m개 neuron의 weighted sum을 동시에 계산
Batch forward	Z = X W^T + b	X: (B,n), W: (m,n) Z: (B,m)	Z = X @ W.T + b	B개 sample을 한 번에 Linear layer에 통과
Backprop error	delta_k = f'(z_k) ⊙ (W_k^T delta_(k+1))	W_k^T: (n,m) delta_(k+1): (m,) delta_k: (n,)	delta = f_prime(z) * (W.T @ delta_next)	다음 layer의 error signal을 현재 layer로 전달
Weight gradient	grad W_k = delta_(k+1) a_k^T	delta: (m,), a: (n,) grad W: (m,n)	dW = np.outer(delta, a)	각 weight가 loss에 미친 변화율을 계산
Bias gradient	grad b = delta	delta: (m,) -> db: (m,)	db = delta	bias parameter의 gradient
Gradient norm	||grad W||_2	grad W: (m,n) -> scalar	g = np.linalg.norm(dW)	vanishing / exploding gradient 진단
Zero ratio	N(a = 0) / N	activation -> scalar	r = np.mean(a == 0)	ReLU 등의 zero activation 비율 관찰

Formula Note. Single Neuron Weighted Sum

수학식: z = w^T x + b | 전개: w^T x = w_1 x_1 + w_2 x_2 + ... + w_n x_n
Shape: w:(n,1), x:(n,1) -> w^T:(1,n) -> (1,n) @ (n,1) = (1,1)
의미: 입력 x와 weight w가 동일한 column-vector shape일 때 w를 전치하여 내적한다. 각 입력 x_i에 대응하는 w_i를 곱한 뒤 모두 합산하므로 weighted sum이 된다. 마지막으로 bias b를 더해 neuron의 선형 입력 z를 만든다.
NumPy 1-D 주의: np.array([1,2,3])의 shape (3,)에는 row/column 방향이 없다. 따라서 w.T의 shape도 (3,)이며 w @ x로 dot product를 계산한다.

Formula Note. Single Neuron to Layer to Batch

범위	수학식	대표 Shape	NumPy
Single neuron	z = w^T x + b	(1,n) @ (n,1) -> scalar	w.T @ x + b
Layer	z = W x + b	(m,n) @ (n,1) -> (m,1)	W @ x + b
Batch	Z = X W^T + b	(B,n) @ (n,m) -> (B,m)	X @ W.T + b